DECA Lab Logbook — Spring Term

Lab 2 | Challenge Idea: Run-Length Encoding Optimisation

Challenge Idea: Run-Length Encoding Optimisation

1. Context and Motivation

The baseline 32-bit multiply routine (Task 5) implements a shift-and-add loop: each iteration tests the LSB of op1, conditionally adds the shifted op2 into the 32-bit accumulator, then shifts both operands. This costs one 32-bit addition for every 1-bit in op1, giving a worst-case of 16 additions for a 16-bit multiplier.

This section proposes a purely algorithmic improvement — requiring no new hardware or new EEP1 instructions — that exploits the structure of consecutive runs of 1-bits in op1 to reduce the number of accumulation operations.

Key insight: A contiguous block of k ones contributes the same partial product as a single addition at the top of the block and a single subtraction at the bottom, regardless of how large k is.

2. Mathematical Foundation

Consider a run of consecutive 1-bits in op1 from bit position L (least significant) to H (most significant), where H ≥ L. The contribution of this run to the product is:

$op2 \times (2^L + 2^{L+1} + \cdots + 2^H)$

This geometric series has the closed form:

$2^L + \cdots + 2^H = 2^{H+1} - 2^L$

Therefore the entire run contributes:

$\boldsymbol{op2 \times (2^{H+1} - 2^L) = (op2 << (H+1)) - (op2 << L)}$

This is the core of the idea: k additions are replaced by 1 subtraction + 1 addition, saving (k − 2) operations per run. For a run of 8 ones this saves 6 additions; for a run of 16 ones (all bits set) it saves 14 additions.

Example: op1 = 0b0000_0000_1111_1100 has a run of six 1-bits from L = 2 to H = 7.

- Baseline: 6 additions (one per 1-bit at positions 2, 3, 4, 5, 6, 7).
- Improved: 1 subtraction ($-op2 \times 2^2 = -op2 \times 4$) + 1 addition ($+op2 \times 2^8 = +op2 \times 256$).
- Net result: op2 × (256 − 4) = op2 × 252. Verification with op2 = 3: 3 × 256 − 3 × 4 = 768 − 12 = 756 = 3 × 252. √

3. Performance Comparison

Scenario	op1 example	Baseline adds	Improved ops	Saving
All ones (best case)	1111 1111 1111 1111	16	2	14 operations

Scenario	op1 example	Baseline adds	Improved ops	Saving
One long run	0001 1111 1110 0000	10	2	8 operations
Two separated runs	0011 1100 0011 1100	8	4	4 operations
Many isolated bits	0101 0101 0101 0101	8	16	−8 (slightly worse)
All zeros (trivial)	0000 0000 0000 0000	0	0	0

The optimisation is most effective when op1 has dense, long runs of ones — common in masks, offsets, and coefficients. It is neutral when all runs have length exactly 1 (isolated bits), and marginally worse for a fully alternating pattern, the theoretical worst case that rarely occurs in practice.

4. Decision

After evaluating this approach, it was not adopted as the main implementation. While mathematically valid and capable of significant improvement for multipliers with long runs of 1s, the benefit is strongly dependent on the input pattern. A fully alternating op1 would actually increase the operation count compared to the baseline.

In addition, the software logic required to detect run boundaries (transitions between 0 and 1) and to correctly handle edge cases introduces significant control overhead, adding extra instructions to each iteration even in the common case.

Radix-4 Booth encoding was selected instead: it delivers a guaranteed 2× reduction in partial products for all 16-bit inputs, regardless of the bit pattern, and is implementable in fixed hardware with a predictable 9-cycle latency.